FAQ-05: How to manage ActiveGate updates on Dynatrace SaaS

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 05 — Managing ActiveGate Updates (SaaS) | **Created:** May 2026 | **Last Updated:** 08/24/2026

Overview

ActiveGate updates differ from OneAgent updates in a way that matters operationally: ActiveGates are infrastructure, not endpoints. A single ActiveGate sits in the path of many OneAgents, hosts a set of extensions, and may run a synthetic browser engine. When an ActiveGate restarts to update, the impact is concentrated — not distributed across thousands of hosts the way a OneAgent restart is.

That concentration is the reason ActiveGate update management deserves a separate decision than OneAgent update management. The mechanics look similar (auto-update toggle, version checking, restart), but the operating practices around sequencing, HA pairs, role-specific validation, and rollback are distinct. It is also why the decision is not purely operational: an ActiveGate is in-path infrastructure with a broad network footprint, and AG security fixes ship inside ordinary version updates — so update policy is a security-posture decision as much as a change-management one (§3).

This FAQ covers: the update mechanism, the auto-update vs manual decision, the sequencing rule (ActiveGates before OneAgents), HA-pair rolling updates, role-specific considerations (routing, synthetic, Extension Framework 2.0, cloud monitoring), validation, rollback, and the most common pitfalls.

> **Scope:** SaaS only. Managed Cluster ActiveGates and the Cluster ActiveGate update model are out of scope here — with one exception: §2 covers the Cluster API v2 `autoUpdate` schema change, because an earlier revision of this document misattributed it to the SaaS Environment API and Managed automators need the correction.

Table of Contents

Prerequisites

Requirement	Details
Audience	Platform team, SRE leads, network/proxy owners, change-management stakeholders
Format	Decision-support document — presents trade-offs and recommendations, no hands-on lab
Deployment	Dynatrace SaaS Environment ActiveGates. Managed Cluster ActiveGate is out of scope.
Related topic series	ONBRD (Dynatrace Onboarding), SYNTH (Synthetic Monitoring), CLOUD (Cloud Provider Integrations)
Related FAQ	**FAQ-04: How to manage OneAgent updates on Dynatrace SaaS** — sequencing depends on AG version

1. Why ActiveGate Updates Need Attention

OneAgent updates are distributed — thousands of hosts, each with its own restart, no single one affecting much. ActiveGate updates are concentrated — one component in the path of many flows, and the restart touches every flow it serves.

The practical consequences:

- **Connectivity gap during restart.** OneAgents and other clients route through the ActiveGate. A restart introduces a brief gap (typically tens of seconds). HA pairs absorb this — single AGs do not.
- **Bundled components update with the AG.** Synthetic browser engine, Extension Framework 2.0 extensions, cloud-monitoring connectors — these don't have independent versions; they ship with the AG version.
- **Version compatibility downstream.** OneAgents connecting through an older AG can encounter feature or protocol mismatches once OneAgents themselves update. Keeping the AG ahead avoids the failure mode.
- **Role-specific behavior changes.** A synthetic-enabled AG restart re-initializes browser monitors; an EF 2.0-enabled AG restart reloads extensions. The "blast radius" of the restart depends on what the AG is configured to do.

Without active management	With active management	Impact
AGs drift behind OneAgent versions	AGs lead, OneAgents follow	Avoids mixed-version protocol issues
Synthetic monitors regress unexpectedly after bundled engine change	Browser-engine change is validated post-update	Monitor false-failures are caught quickly
EF 2.0 extensions fail to reload after a restart, ingest stops silently	Extension reload is part of post-update validation	Stays current on extension behavior
Single-AG architectures take observability gaps during every update	HA pair, rolled one at a time	No observability gap during update

In community practice, the most consistent benefit teams report from disciplined AG update management is "we stopped having synthetic monitors flake mysteriously after sprint updates" — the symptom that surfaces when bundled engine changes happen invisibly. Verify against your own monitor history.

> **Sources:** Update ActiveGate (DT docs) — *"When a new version is available, the installation package is downloaded and installed automatically"* (re-read at source 08/24/2026; the page has been reworded since this entry was written, and the longer sentence previously quoted here no longer appears on it — the behaviour described is unchanged); availability check runs at ~30-minute intervals.

2. How ActiveGate Updates Work on SaaS

ActiveGate update flow on SaaS:

1. The ActiveGate checks Dynatrace SaaS for a newer version on an interval (~30 minutes).
2. If a newer version is available and auto-update is enabled for this AG, the new installer is downloaded.
3. The new version is installed and the AG service restarts on it.
4. Routes, extensions, and synthetic engine re-initialize on the new version.
5. The AG re-registers and resumes serving traffic.

A few mechanics worth knowing:

- **Settings are per-ActiveGate.** Unlike OneAgent — where host-group is the natural grouping — ActiveGates are managed individually. Each AG has its own update setting. There is no host-group equivalent for AGs.
- **Auto-update can be disabled.** When disabled, a *one-click "Update now"* control appears in the AG's settings when a new version is available.
- **The check interval is fixed.** Roughly every 30 minutes; this is a platform behavior, not a configurable knob.
- **The restart window is short.** Tens of seconds typically. HA pair architectures absorb this; single-AG architectures briefly drop traffic.

> **Sources:** Update ActiveGate (DT docs) — describes per-AG settings, one-click update, the 30-minute availability check; *"Go to Settings to open the ActiveGate updates settings for the particular ActiveGate."*

A cautionary note on `targetVersion` and `updateWindows`

An earlier revision of this document reported that AG auto-update had gained `targetVersion` (pin the version AGs update to) and `updateWindows` (constrain *when* updates run) as API-configurable properties. That reporting had two defects worth correcting explicitly, because both change what an automator should do.

First, the API family was wrong. Those properties live on the **Cluster API v2 (Dynatrace Managed)** endpoints — `GET|PUT /activeGates/autoUpdate`, `POST /activeGates/autoUpdate/validator`, and the three per-AG `/activeGates/{agId}/autoUpdate` equivalents. They were never part of the SaaS Environment API. **On SaaS, the per-ActiveGate settings surface described above remains the control surface** — there is no SaaS API knob to pin a target version or set an update window in its place.

Second, the addition was reversed. API 1.344 (published 07/15/2026, rollout from 07/29/2026) **removes `targetVersion` and `updateWindows` again** from all six of those Cluster API v2 endpoints, and also changes the read-only status of properties on the per-AG endpoints. If you automate Dynatrace Managed ActiveGate updates, **remove both properties from your request bodies before the change reaches your cluster** — a `PUT` that still sends them is a request built against a schema that no longer accepts them. Verify against your own cluster's API version rather than assuming the rollout date.

The general lesson generalises past this one property pair: a capability announced in a sprint's release notes can be withdrawn in the next one. That is the reason this series names the version on every release-tied claim and keeps the pre-version guidance in place rather than deleting it — here, the pre-version guidance (per-AG settings on SaaS) turned out to be the guidance that survived.

> **Fleet Management.** A centralized console for OneAgent, ActiveGate, and network-zone management at scale, with purpose-built ActiveGate views for deployment health and upgrade scheduling, was introduced around SaaS 1.343 (July 2026). **Its documentation page still carried a "Coming soon" banner when checked in July 2026**, so verify current GA status and feature scope against the Fleet Management documentation (DT docs) and your tenant's release notes before planning around it — availability may vary. Once available in your tenant, fleets beyond a handful of AGs should plan updates from Fleet Management rather than per-AG settings pages; until then, per-AG settings remain the working model.

> **Sources:** API changelog 1.342 (DT docs) — `targetVersion` / `updateWindows` added to the Cluster API v2 ActiveGate `autoUpdate` schemas, API changelog 1.344 (DT docs) — both properties removed again across the six endpoints; per-AG read-only status changed, SaaS 1.343 release notes (DT docs), Fleet Management (DT docs) — "Coming soon" banner as checked 07/08/2026. **Derived:** the "remove both properties from request bodies before the change lands" instruction follows from the 1.344 schema removal plus the staged-rollout model.

3. The Auto-Update vs Manual Decision

Two effective modes per ActiveGate:

Mode	What it does	When to pick it
Auto-update enabled (default)	AG downloads and installs new versions as they become available.	Most ActiveGates — especially routing AGs in HA pairs, where the rolling-update pattern absorbs the restart.
Auto-update disabled	AG checks for new versions but does not install. A *one-click "Update now"* control appears when a new version is available.	AGs where the restart window or bundled-component change needs to be deliberately scheduled — single-AG sites without HA, AGs with high-stakes EF 2.0 extensions, AGs running synthetic monitors against revenue-critical workflows.

Decision factors

- **HA pair vs single AG.** HA pairs make auto-update low-risk: roll one at a time, the partner absorbs the load. Single AGs cause a connectivity gap during restart — disabling auto-update lets you schedule that gap. In community practice, single-AG architectures are usually a temporary state on the path to HA — the right long-term answer is *deploy a second AG*, not *disable auto-update*.
- **Security releases and your vulnerability-remediation SLA.** This is the factor most often missing from the conversation, because the rest of the decision reads as purely operational. An ActiveGate is **in-path infrastructure with a broad network footprint** — it terminates OneAgent connections, holds credentials for cloud connectors and extensions, reaches into monitored networks, and often sits in a DMZ or a routable segment by design. That footprint is why an AG security fix is not the same class of

change as a feature update. And AG security fixes do ship inside ordinary version updates rather than on a separate channel: **ActiveGate 1.343 shipped a security upgrade addressing CVE-2026-40984 and CVE-2026-40983**, delivered as part of the normal version, so an AG left behind on 1.342 stays exposed until it takes its next update.

The test to apply: **every AG on manual updates needs a demonstrable path to apply a security release inside your organization's vulnerability-remediation SLA.** Not a hope that someone notices — a named owner, a trigger, and a window that fits inside the SLA clock. If the manual process cannot meet that SLA — and for a single AG whose restart requires a change ticket, it usually cannot — **the answer is HA plus auto-update, not manual updates.** HA is what removes the restart-window objection that motivated manual mode in the first place, which makes it the fix for both problems at once. Manual mode is defensible for a scheduled feature update; it is much harder to defend for a security release.

- **Synthetic load.** A synthetic-heavy AG carries a browser engine version that changes with AG version. If your synthetic monitors are revenue- or SLO-load-bearing, disabling auto-update on the synthetic AG lets you validate the new browser engine before it goes live against production monitors.
- **EF 2.0 extension load.** AGs hosting EF 2.0 extensions reload all extensions on restart. If you have many or complex extensions, disabling auto-update lets you stage the restart at a known time.
- **Cloud connectors.** AGs running AWS/Azure/GCP cloud-monitoring connectors are usually safe to auto-update — connector behavior is stable across versions — but a connector-heavy AG benefits from a known restart time for the same reason.
- **Network change-control.** Some networks treat any in-path infrastructure restart as a change requiring a ticket. Auto-update conflicts with that model; pick manual — but read the security factor above before settling there, because change-control and a remediation SLA usually come from the same governance function and are supposed to be reconciled rather than traded off.

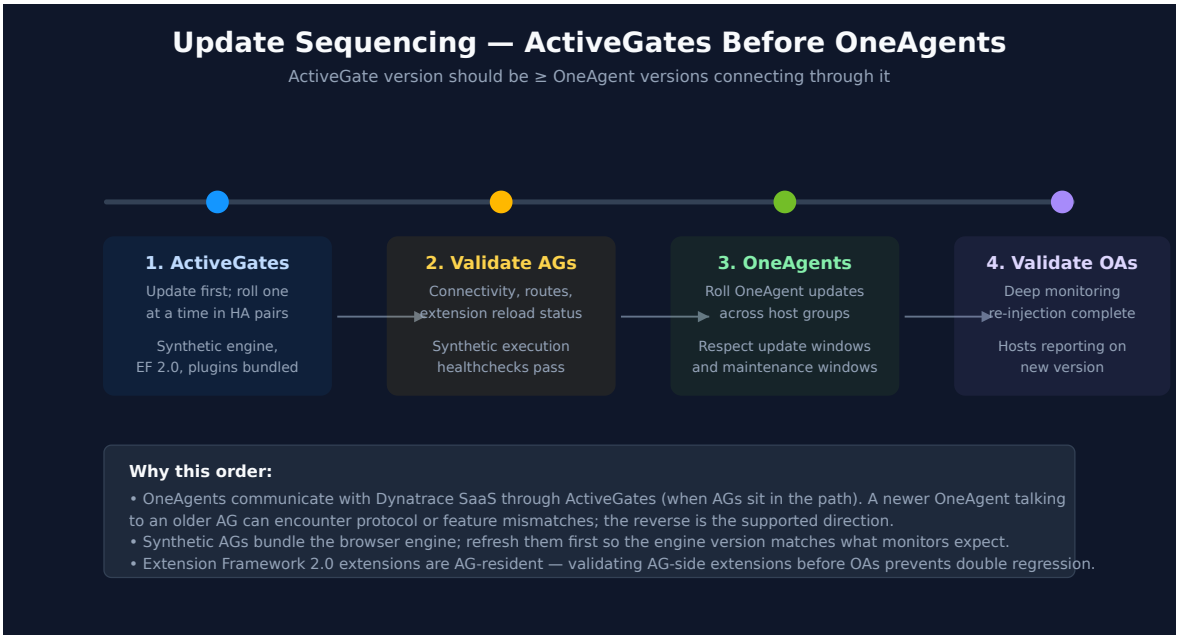
> **Sources:** Update ActiveGate (DT docs) — auto-update toggle is per-AG; one-click *"Update now"* available when auto-update is disabled and a new version is ready, ActiveGate 1.343 release notes (DT docs) — security upgrade addressing CVE-2026-40984 and CVE-2026-40983, shipped inside the ordinary version update. **Derived:** the "if the manual process cannot meet the SLA, the answer is HA plus auto-update" conclusion combines the security-fix delivery model with the per-AG restart behavior that HA absorbs.

4. Sequencing — ActiveGates Before OneAgents

The operating rule:

> **Update ActiveGates first. Then update OneAgents.**

The asymmetry: a newer ActiveGate accepting traffic from older OneAgents is the supported direction. A newer OneAgent talking to an older AG can encounter feature or protocol mismatches that the platform doesn't aggressively defend against.



What "first" means in practice

- **Sprint cadence:** On most SaaS sprint cycles, ActiveGates auto-update first because they check more frequently and react sooner — the natural order is usually correct without explicit coordination. The deliberate sequencing matters when *manual* updates are involved (disabled auto-update on either side).
- **Major version bumps:** When a major OneAgent version ships with corresponding AG features, the order is explicit: schedule the AG update window first, complete it, validate, then schedule the OneAgent window.
- **Mixed environments:** If you have both auto-updating and manually-updated AGs, the manual ones are the constraint — they bound how fast the AG tier as a whole advances.

If your tenant uses no private ActiveGates (OneAgents connect directly to Dynatrace SaaS), this sequencing concern doesn't apply for the routing path. It still applies for any **specialized** AGs (synthetic, EF 2.0, cloud-monitoring) that produce data the rest of the platform consumes.

Cross-reference: FAQ-04: How to manage OneAgent updates on Dynatrace SaaS for the OneAgent-side of the same problem.

> **Sources:** Update ActiveGate (DT docs), OneAgent update (DT docs). **Derived:** the AG-before-OneAgent rule is community / engagement guidance grounded in the asymmetric compatibility direction across both update pages; neither page states it as a single explicit rule.

5. HA Pairs and Rolling Updates

HA pair architecture is the operating norm for any ActiveGate role serving OneAgent traffic, synthetic, or cloud monitoring in a non-toy environment. The update pattern is:

1. **Roll one AG at a time.** Update the first AG; wait for it to fully re-register and resume serving traffic.
2. **Validate** before touching the second AG. Routes registered, extensions reloaded, no error spikes on the AG itself or on dependents.
3. **Update the second AG.**

This pattern is mostly automatic when auto-update is on across both AGs — the 30-minute check interval and the natural restart-staggering between them tends to produce the rolling pattern without explicit coordination. The pattern needs to be deliberate when:

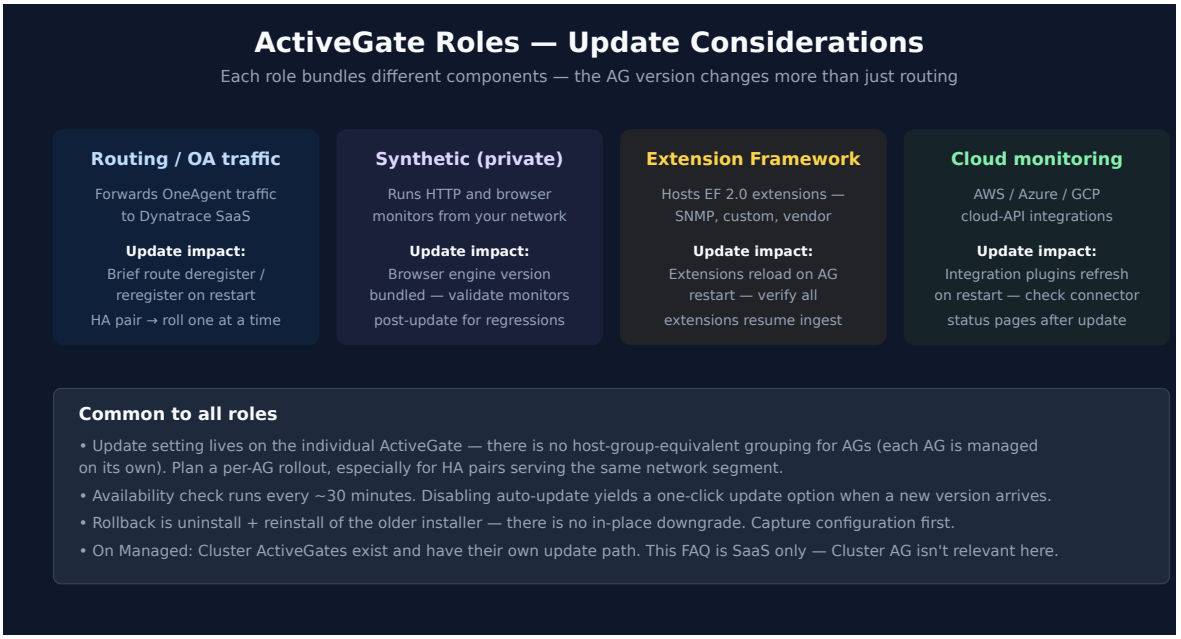
- You've disabled auto-update on both AGs and are running manual updates → roll them yourself, one at a time.
- The AGs are on different sprint update windows by design → the staggering is built in, just confirm it.
- You're rolling out a known-risky update (major version, big extension framework change) → validate the first AG fully before the second.

Why not both at once: during the brief restart window, an AG isn't serving traffic. If both restart simultaneously, you have a connectivity gap. HA pairs only deliver no-gap operation when at least one of them is up.

> **Sources:** Update ActiveGate (DT docs). **Derived:** rolling-update sequencing for HA pairs is community / engagement practice — the docs describe per-AG updates but not the operating discipline of staggering them; the discipline follows from the per-AG restart behavior plus general HA-pair principles.

6. Roles and Update Implications

ActiveGate roles bundle different components — and the update affects each role's components differently.



Routing / OneAgent traffic

The "default" role. Restart causes a brief route deregister/reregister. In an HA pair the partner absorbs the traffic. For single AGs, this is the role most affected by the restart window.

Synthetic (private locations)

Private synthetic locations are hosted by ActiveGates with the synthetic role enabled. The browser engine ships with the AG version. After an update, browser monitors may behave differently — usually fine, occasionally a monitor that depended on a specific browser behavior surfaces a failure.

Post-update validation: re-run a representative sample of browser monitors before the next scheduled execution, and check that HTTP monitors continue to pass. Synthetic-flake immediately after an AG update is *not* uncommon and is almost always the engine change, not the monitored site.

Extension Framework 2.0

EF 2.0 extensions are AG-resident. On AG restart, all extensions on that AG reload. Extension behavior is generally stable across AG versions, but the reload itself can surface configuration issues — an extension that was running with a stale config gets the new config on reload and starts misbehaving.

Post-update validation: check that all expected extensions list as `RUNNING` (or equivalent) post-restart and that their ingest streams (custom metrics, logs, events) resume within the expected interval.

Cloud monitoring

AWS, Azure, and GCP integration connectors are bundled with the AG. Connector behavior is stable across versions in practice, but new versions occasionally bring new resource-type coverage or change pagination defaults. Post-update validation: connector status pages in the Cloud app (or equivalent) show healthy connectors, and the data lag for cloud-imported metrics matches the pre-update baseline.

Common across all roles

- Settings are per-AG. There's no role-grouping for update settings.
- Auto-update availability check runs every ~30 minutes.
- Rollback is uninstall + reinstall of the older installer — no in-place downgrade.

> **Sources:**
>
> - Update ActiveGate (DT docs) — per-AG update mechanic, one-click update, availability check interval.
> - The per-role validation guidance is community / engagement-derived — Dynatrace docs describe each role's setup but do not document a per-role post-update validation checklist.

7. Validation After Update

Post-update validation for ActiveGates is more concentrated than for OneAgents — fewer entities, but more roles bundled into each.

Core validation set

1. **AG version reported.** The AG status page reflects the new version. To check the whole fleet at once in DQL, query the Smartscape node — this is the only working DQL path, because **there is no classic ActiveGate entity type**: `dt.entity.active_gate`, `dt.entity.environment_active_gate`, and `dt.entity.environment_activegate` all fail with *"The entity type ... wasn't found"*, so `fetch dt.entity.*` was never an option here. Use:

```
dql smartscapeNodes "ACTIVEGATE" | fields id, name, dt.active_gate.version
```

The node also carries `dt.active_gate.group.name`, `dt.network_zone.id`, `is_containerized`, `is_fips`, `modules[]`, and `os.type` — enough to segment the fleet by role and topology in the same query. If you need the pre-Smartscape path (older tenants, or automation predating it), the classic **Entities API v2** selector `GET /api/v2/entities?entitySelector=type("ENVIRONMENT_ACTIVE_GATE")` is a *different surface* from DQL and may still respond; that it works says nothing about the DQL entity type, which never existed. See FAQ-16 §2 for the full classic-to-Smartscape mapping and why ActiveGate is the odd row in it.
2. **Routes registered.** Traffic resumes flowing through the AG. No host-side "lost connection to ActiveGate" event spike.
3. **Extensions reloaded.** All EF 2.0 extensions show `RUNNING`. Ingest streams from each extension resume within their expected interval.
4. **Synthetic monitors functioning.** If the AG hosts a private synthetic location, run a representative monitor manually and confirm pass. Watch the next scheduled execution.
5. **Cloud connectors connected.** If the AG runs cloud monitoring, the connector status pages show healthy and the data lag for cloud metrics has returned to baseline.

Validation timing

- **Routing-only AGs:** validation can be near-instant — traffic either flows or it doesn't.
- **Synthetic AGs:** allow at least one full monitor execution cycle before declaring success. Some browser regressions only appear under load.

- **EF 2.0 AGs:** allow at least one full extension scrape cycle (typically 1 minute for fast extensions, 5–15 minutes for slow ones) before declaring success.
- **Cloud monitoring AGs:** allow at least one cloud-API poll cycle (typically 1–5 minutes) plus normal ingestion lag.

For change-controlled environments, document the validation set and timing as part of the change ticket so the post-update verification is auditable.

A note on what the version query will show you. Running the query above across a fleet is also the fastest way to see version skew — on the validation tenant all four ActiveGates reported `1.341.5`, i.e. two releases behind the then-current 1.343, which is the ordinary steady state for a fleet on its own upgrade schedule rather than a fault. Read the result as a fleet inventory, not a pass/fail check.

> **Sources:** Update ActiveGate (DT docs), Entities API v2 — GET entities (DT docs). The `smartscapeNodes "ACTIVEGATE"` query and the three failing `dt.entity.*active_gate*` spellings were executed against a live Dynatrace tenant, 07/30/2026 — 4 nodes returned, 0 bytes scanned. **Derived:** the validation checklist combines the documented update mechanic with role-specific operating practice; it is not a single documented checklist.

8. Rollback Considerations

ActiveGate rollback is less common than OneAgent rollback (fewer AGs, more deliberate updates), but the mechanic is similar:

- **Rollback = uninstall + reinstall older installer.** No in-place downgrade.
- **Capture configuration first.** AG configuration files (`*.properties` under the AG install directory), enabled features, extension installations, connection endpoints, and any custom certificates need to be restored on the older install. Take a backup before uninstalling.
- **Auto-update will revert your rollback.** If you reinstall an older version with auto-update enabled, the AG will update back to current on its next check. Disable auto-update on the rolled-back AG until the issue is resolved.
- **HA partner consideration.** If you're rolling back one AG in an HA pair because of a version-specific issue, the partner is still on the new version. Mixed-version HA pairs are tolerated for short periods but are not a long-term state.

- **Extensions and synthetic monitors may need re-validation.** Rolling back the AG version rolls back the bundled browser engine and EF 2.0 runtime — same considerations as forward updates apply in reverse.

In community practice, rollback is usually a containment move while the underlying issue is investigated. The expected resolution path is *fix forward* — patch from Dynatrace, configuration adjustment, or extension fix — rather than long-term rollback. Plan rollback as a 24–72 hour state, not a steady state.

> **Sources:** Update ActiveGate (DT docs). **Derived:** the rollback playbook combines the documented uninstall/reinstall mechanic with general infrastructure-change-management practice — community / engagement guidance.

9. Common Pitfalls

Pitfall	Why it happens	What to do instead
Updating OneAgents before ActiveGates.	Natural intuition: agents first, infrastructure last.	Reverse it — AGs first, OneAgents second. The compatibility direction is asymmetric.
Updating both AGs in an HA pair simultaneously.	Auto-update on both, no staggering enforced; or a manual change-window applied to both at once.	Roll one at a time. Validate before touching the second. HA-pair updates need to be staggered.
Treating synthetic monitor flake post-update as a real failure.	A monitor that was relying on a specific browser behavior surfaces a failure after the bundled engine version changes.	Validate monitor behavior after the AG update before declaring an outage. Browser-engine changes are usually the cause of immediate-post-update synthetic flake.
Forgetting that EF 2.0 extensions reload on AG restart.	Extension was running with a stale config that worked-with-the-old-version; the new version reloads and surfaces the config issue.	Treat extension reloads as part of the AG-update change window. Validate extension status post-restart.

Pitfall	Why it happens	What to do instead
Disabling auto-update on a single AG and forgetting — leaving a known vulnerability unpatched on in-path infrastructure.	"We'll update manually." Nobody does. The cost is usually described as version drift, which understates it: AG security fixes ship *inside* ordinary version updates (ActiveGate 1.343 carried the fix for CVE-2026-40984 and CVE-2026-40983), so a forgotten AG is not merely behind on features — it holds an open vulnerability window on a component that terminates agent connections and holds connector credentials, for as long as nobody notices.	Either run HA so you can leave auto-update on — the recommended answer — or pair manual mode with a named owner, a calendar mechanism, **and** a check that the manual path fits inside your vulnerability-remediation SLA (§3). If it does not fit, that is the signal to deploy the second AG rather than to tighten the reminder.
Trusting the 30-minute check to be exact.	Update windows planned around an assumed exact check time.	The check is *approximate* — design your window with margin (30 minutes is the lower bound, not a deterministic schedule).
Not validating cloud connectors after an update.	Connector behavior is usually stable; teams skip the check.	Glance at connector status pages and ingest-lag for cloud metrics post-update — it's a 60-second check that catches the rare regression.
Automating Managed AG auto-update against `targetVersion` / `updateWindows`.	Both properties were added to the Cluster API v2 `autoUpdate` endpoints in 1.342 and read as a stable capability.	API 1.344 removes both again. Strip them from request bodies before the change reaches your cluster (§2). On SaaS the properties never existed — per-AG settings are the control surface.

Sources:

- Update ActiveGate (DT docs) — update mechanic and check interval.
- ActiveGate 1.343 release notes (DT docs) — security upgrade addressing CVE-2026-40984 and

> CVE-2026-40983.
>
> - API changelog 1.344 (DT docs) — `targetVersion` / `updateWindows` removed from the Cluster API v2 ActiveGate `autoUpdate` endpoints.
> - The remaining items are community / engagement-derived patterns — observed across fleets to be worth flagging, not formally documented as anti-patterns.

10. Recommended Approach

For most SaaS tenants with private ActiveGates, the right configuration is:

1. **Deploy ActiveGates in HA pairs** for any role that is in the path of OneAgent traffic, synthetic monitors, or cloud monitoring. Single-AG architectures are a transitional state, not a target — and HA is what makes leaving auto-update on defensible, which is why it is item 1 rather than a nice-to-have.
2. **Leave auto-update enabled** on routing-only AGs in HA pairs. The natural staggering plus partner-absorbs-load makes auto-update low-risk.
3. **Disable auto-update on synthetic-heavy AGs** where monitor reliability is load-bearing for SLO or revenue. Schedule synthetic-AG updates during low-traffic windows and validate browser-monitor behavior before the next monitor execution cycle.
4. **Disable auto-update on EF 2.0-heavy AGs** if you have many or complex extensions. Schedule the restart at a known time so extension reloads are not a surprise.
5. **For every AG you set to manual (items 3 and 4), name an owner and confirm the manual path fits inside your vulnerability-remediation SLA** — security fixes arrive inside ordinary AG version updates, so a manual AG with no owner is an open vulnerability window, not just a stale version (§3, §9). Where the SLA cannot be met, revisit items 3 and 4: the answer is HA plus auto-update, not a tighter reminder.
6. **Update ActiveGates before OneAgents** as standard practice. See FAQ-04 for the OneAgent-side discussion.
7. **Roll HA pairs one at a time.** Validate the first AG before touching the second.
8. **Validate per-role after every update** — routes, extensions, synthetic monitors, cloud connectors — at the right cadence for each role. `smartscapeNodes "ACTIVEGATE"` gives you the fleet-wide version inventory in one query (§7).
9. **Plan rollback as a 24–72 hour containment**, not a steady state. Disable auto-update on rolled-back AGs to prevent revert — and put the rollback on the same SLA clock as item 5, since a rolled-back AG is by definition on an older version.

For tenants with no private ActiveGates:

- The sequencing concern doesn't apply for routing.
- The role-specific concerns still apply for any specialized AGs (synthetic, EF 2.0, cloud monitoring) you do run.
- The HA-pair pattern still applies to specialized AGs in the same way.

Summary

ActiveGate update management is operationally distinct from OneAgent update management — fewer AGs, more roles bundled into each, more concentrated restart impact. The right defaults differ by role: routing AGs in HA pairs auto-update fine; synthetic and EF 2.0 AGs benefit from manual scheduling, provided a named owner keeps them inside the organization's vulnerability-remediation SLA; sequencing AGs before OneAgents is the supported direction across all roles. The most common failure modes are simultaneous updates of HA pair members, OneAgent-before-AG sequencing, and disabled auto-update without a calendar mechanism — the last of which is a security exposure, not merely version drift, because AG security fixes ship inside ordinary version updates.

Next Steps

- Inventory your ActiveGates by role, HA topology, and current version — `smartscapeNodes "ACTIVEGATE" | fields id, name, dt.active_gate.version` (§7).
- Convert any single-AG architectures serving production roles to HA pairs.
- Set auto-update intentionally on each AG based on role (not "on everywhere" or "off everywhere"), and record an owner for every AG left on manual.
- Check the manual-mode AGs against your vulnerability-remediation SLA; where the SLA cannot be met, plan the second AG.
- If you automate Dynatrace Managed AG auto-update, remove `targetVersion` and `updateWindows` from request bodies ahead of the API 1.344 schema change (§2).
- Confirm AG sprint cadence is at-or-ahead-of OneAgent cadence; adjust if OneAgents are leading.
- Read **FAQ-04** for the OneAgent-side of the same problem, and **FAQ-16 §2** for why ActiveGate is the odd row in the classic-to-Smartscape mapping.

- Document AG roles, HA topology, and update policy alongside your host-group naming (see **FAQ-01**) and tagging strategy (see **FAQ-02**).
